

11.[Open Ended- SCAN Algorithm]

Consider a disk queue that contains requests for input/output operations at cylinder numbers **98, 183, 41, 122, 14, 124, 65, and 67**. The disk head is initially positioned at **cylinder 53**, and the cylinders are numbered from **0 to 199**. Use the **SCAN (Elevator) disk scheduling algorithm** to service these requests by moving the disk head in a particular direction and then reversing its direction after reaching the end. Write a program to determine the **total head movement**, measured in the number of cylinders traversed, while processing all the given requests

CODE :

```
~
GNU nano 8.7
#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {
    int i, j, temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int request[] = {98, 183, 41, 122, 14, 124, 65, 67};
    int n = 8;
    int head = 53;
    int disk_size = 200;

    int i, total_movement = 0;
    int left[20], right[20];
    int l = 0, r = 0;

    // Divide requests into left and right of head
    for(i = 0; i < n; i++) {
        if(request[i] < head)
            left[l++] = request[i];
        else
            right[r++] = request[i];
    }

    // Add end points
    left[l++] = 0;
    right[r++] = disk_size - 1;

    // Sort both arrays
    sort(left, l);
    sort(right, r);

    printf("Seek Sequence: ");

    // Move towards right first
    for(i = 0; i < r; i++) {
        printf("%d ", right[i]);
    }
}
```

```

GNU nano 8.7
int main() {
    int request[] = {98, 183, 41, 122, 14, 124, 65, 67};
    int n = 8;
    int head = 53;
    int disk_size = 200;

    int i, total_movement = 0;
    int left[20], right[20];
    int l = 0, r = 0;

    // Divide requests into left and right of head
    for(i = 0; i < n; i++) {
        if(request[i] < head)
            left[l++] = request[i];
        else
            right[r++] = request[i];
    }

    // Add end points
    left[l++] = 0;
    right[r++] = disk_size - 1;

    // Sort both arrays
    sort(left, l);
    sort(right, r);

    printf("Seek Sequence: ");

    // Move towards right first
    for(i = 0; i < r; i++) {
        printf("%d ", right[i]);
        total_movement += abs(head - right[i]);
        head = right[i];
    }

    // Then move towards left
    for(i = l - 1; i >= 0; i--) {
        printf("%d ", left[i]);
        total_movement += abs(head - left[i]);
        head = left[i];
    }

    printf("\nTotal Head Movement: %d\n", total_movement);

    return 0;
}

```

OUTPUT :

```

rachn@LAPTOP-6NDD8TJ3 MINGW64 ~
$ cd ~/Desktop/CD24036_OS/Practical11
-bash: cd: /home/rachn/Desktop/CD24036_OS/Practi

rachn@LAPTOP-6NDD8TJ3 MINGW64 ~
$ nano scan.c

rachn@LAPTOP-6NDD8TJ3 MINGW64 ~
$ gcc scan.c -o scan

rachn@LAPTOP-6NDD8TJ3 MINGW64 ~
$ ./scan
Seek Sequence: 65 67 98 122 124 183 199 41 14 0
Total Head Movement: 345

rachn@LAPTOP-6NDD8TJ3 MINGW64 ~
$

```